

PRÁCTICA 1: Programar un simulador de un autómeta con pila

Objetivo

El objetivo de la práctica consiste en programar un simulador de un autómeta con pila, realizando un diseño orientado a objetos.

Entrega

La ejecución de la práctica será revisada en la sesión de entrega en el laboratorio.

La entrega se realizará a través del aula virtual:

Es necesario proporcionar el código fuente en su totalidad, incluyendo todos los archivos y librerías indispensables para su compilación y ejecución. Se incluirá un archivo README.

Fecha límite de entrega: 9 de octubre de 2026.

Enunciado

En la práctica se puede desarrollar el autómeta de pila con finalización por vaciado de pila (APv) o por estado final (APf) (únicamente uno de los dos tipos) [1]. En el archivo README se debe indicar cuál es el tipo implementado.

Los elementos del autómeta con pila se introducirán en tiempo de ejecución del programa utilizando un fichero de texto con el siguiente formato:

Comentarios

q1 q2 q3 ... # conjunto Q

a₁ a₂ a₃ ... # conjunto $\Sigma^{(*)1}$

A₁ A₂ A₃ ... # conjunto $\Gamma^{(*)1}$

q1 # estado inicial

A₁ # símbolo inicial de la pila

q2 q3 # conjunto F (solamente en el caso de APf)

q1 a^{(*)2} A₁ q2 A^{(*)3} # función de transición: $(q2, A) \in \delta(q1, a, A_1)$

... # cada una de las transiciones en una línea distinta

(*)1 Los símbolos contenidos en Σ y en Γ estarán formados por un único carácter.

(*)2 a es un símbolo de $(\Sigma \cup \epsilon)$. En el fichero el símbolo ϵ se representará por un punto (.)

(*)3 A puede ser ϵ (que se escribirá también en el fichero mediante un punto) o estar formado por uno o más símbolos de Γ , que se escribirán sin espacios en blanco. Por ejemplo: $A = A_1 A_1 A_1$

El programa debe funcionar con los ejemplos descargados del aula virtual y debe verificar que la información proporcionada en el fichero cumple con las restricciones de la definición formal de un autómata, por ejemplo: $s \in Q$.

Una vez leída la definición del autómata, debe ser posible comprobar varias cadenas de entrada e indicar si pertenecen al lenguaje reconocido por el autómata. Las diferentes cadenas de entrada podrán ser introducidas por teclado o por fichero (no es necesario permitir los dos métodos).

Como salida, el programa debe indicar si la palabra pertenece al lenguaje que reconoce el autómata de pila.

El programa tendrá la opción de ejecutar en *modo traza*, donde se mostrará la información del autómata después de aplicar cada transición (estado, cadena, pila y posibles transiciones a aplicar). La traza puede mostrarse por pantalla o escribirse a un fichero (no es necesario permitir los dos métodos).

Notas de implementación

1. Los lenguajes de programación permitidos son C++ o Java.
2. Se debe utilizar un diseño orientado a objetos.
3. Para organizar el proyecto se va a utilizar una estructura estándar [2].
4. La compilación y ejecución completa del programa se debe realizar en consola (no se debe implementar un entorno gráfico).
5. El programa principal aceptará las siguientes opciones por línea de comandos:
 - a. `-config <f>`, *f* es un fichero de texto con la configuración del AP.
 - b. `-trace <y|n>`, indica si se muestra o no la traza durante la ejecución.
 - c. `[-in <f>]`, *f* es un fichero de texto con las cadenas de entrada a comprobar. Opcional. Si no se incluye, las cadenas de entrada se introducirán por teclado.
 - d. `[-out <f>]`, *f* es un fichero de texto donde se almacenará el resultado de la traza. Opcional. Si no se incluye la traza se mostrará por pantalla.

Referencias

[1] Material de clase [Moodle]:
<https://campusvirtual.ull.es/2627/ingenieriytecnologia/course/view.php?id=2627090029>

[2] Organizar un Proyecto C++:
<https://www.studyplan.dev/cmake/organizing-a-cpp-project>